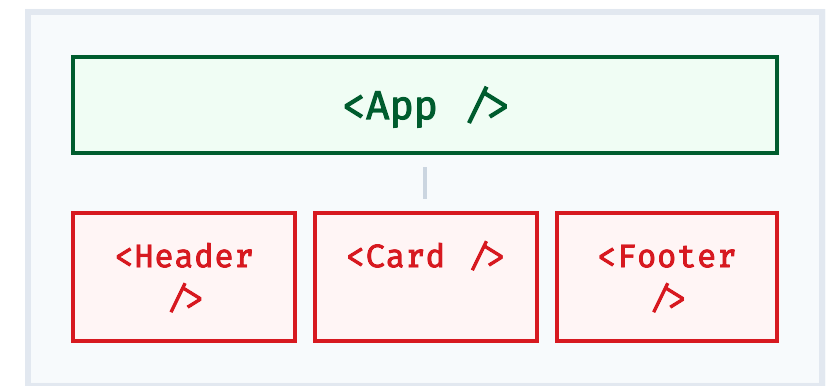


Aula 2: Do HTML Semântico à Interatividade Real

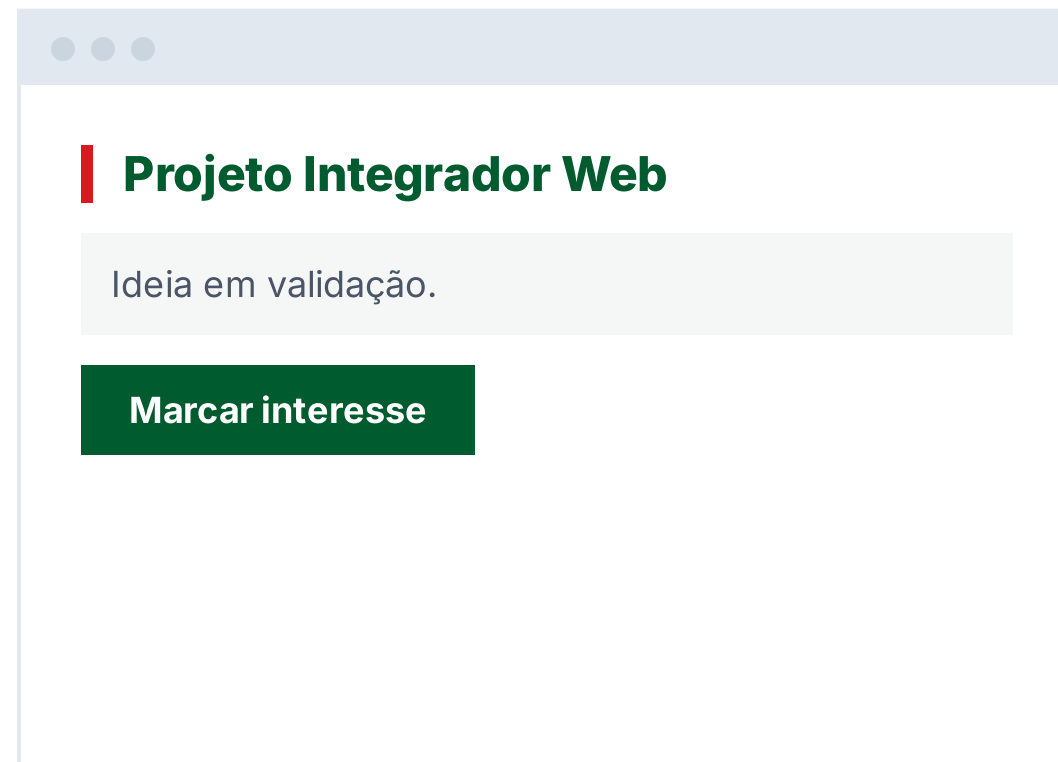
Frameworks, Vite e Componentização:
organizando a interface para crescer.



Arquitetura baseada em componentes

| Retomada: Aula 1 em 4 minutos

- 1 Quem solicita os arquivos e quem responde a requisição?
- 2 Qual é o papel fundamental de HTML, CSS e JavaScript?
- 3 O clique no botão alterou o arquivo HTML ou o DOM em memória?



Protótipo funcional produzido na Aula 1

Roteiro dos 100 minutos

0 – 10 MIN



Abertura e Missão

Retomada da Aula 1 e apresentação do desafio de hoje.

10 – 30 MIN



Ecosistema Moderno

Frameworks, Vite, Vue 3 e o conceito de Componentes.

30 – 85 MIN



Laboratório Prático

Criação, limpeza e componentização do protótipo.

85 – 100 MIN



Entrega e Ticket

Revisão por pares, commit final e avaliação individual.

🎯 **Meta de hoje: Projeto Vite executando com interface componentizada e commit local.**

Desafio: Quando a interface cresce

O Modelo "Tudo em Um" (Aula 1)

```
<!-- index.html -->
<header> ... </header>
<main>
  <section id="projeto">
    <h2>Problema</h2>
    <p>Descrição ... </p>
    <button>Interesse</button>
  </section>
  ... (mais 500 linhas)
</main>
<footer> ... </footer>
```

Arquivos gigantes e responsabilidades misturadas.

Dificuldades da Escala

-  **Manutenção:** Alterar um detalhe exige navegar por centenas de linhas.
-  **Reuso:** Difícil repetir o mesmo padrão visual em outras páginas.
-  **Colaboração:** Conflitos constantes ao editar o mesmo arquivo.

"Concentrar tudo em poucos arquivos torna a evolução do software insustentável."

Solução: Frameworks e Componentização

O QUE É UM FRAMEWORK?

Uma estrutura de suporte (esqueleto) que fornece ferramentas e padrões prontos para acelerar o desenvolvimento.

COMPONENTIZAÇÃO

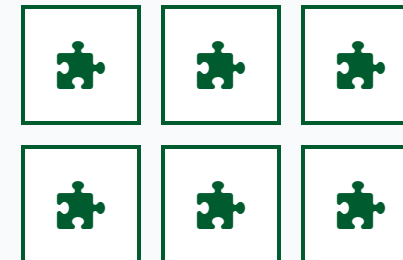
A estratégia de "dividir para conquistar": quebrar a interface em peças independentes, reutilizáveis e fáceis de manter.

💡 **Menos repetição, mais organização.**

ANTES: MONOLITO



AGORA: COMPONENTES

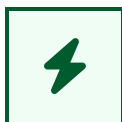


| O Framework: Vue.js 3



Simplicidade Progressiva

Curva de aprendizado suave. Você começa com o básico e avança conforme a necessidade do projeto.



Reatividade Nativa

O Vue cuida da sincronia: quando os dados mudam, a interface atualiza automaticamente sem refresh.



Foco em Componentes

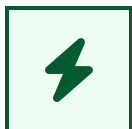
Tudo é um componente. Isso facilita a organização, o reuso de código e a manutenção a longo prazo.



⚠ NÃO CONFUNDA:

O **Vue** é o motor (framework) que faz a página funcionar. O **Vite** é a garagem (ferramenta) que organiza o trabalho.

| A Ferramenta: Vite



Início Instantâneo

Não compila todo o projeto antes de começar.
O servidor sobe em milissegundos.



HMR Ultrarrápido

Hot Module Replacement: a página atualiza apenas o que mudou, sem perder o estado.



Foco no Fluxo

Organiza arquivos, resolve caminhos e prepara o código para o navegador.

```
$ npm run dev

VITE v5.4.0 ready in 128 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose

press h + enter to show help
```

Vite: o motor do desenvolvimento moderno

O Ecossistema: Node.js e npm

Node.js

O motor que executa JavaScript no computador (servidor), não apenas no navegador.

- ✓ Base para ferramentas como Vite.
- ✓ Permite automação de tarefas.
- ✓ Requisito: Versão 20.19+ ou 22.12+.

```
$ node -v  
v22.13.0
```

npm

O gerente que organiza as bibliotecas e scripts do seu projeto.

- ✓ Instala o Vue e o Vite.
- ✓ Gerencia o arquivo package.json.
- ✓ Executa o servidor de dev.

```
$ npm -v  
10.9.2
```


| Single File Components (SFC)

O QUE É?

Um padrão de arquivo (.vue) que encapsula toda a estrutura, lógica e estilo de um componente em um único lugar.

- ✓ **Coesão:** Código relacionado fica junto.
- ✓ **Manutenção:** Fácil de localizar e editar.
- ✓ **Escopo:** O estilo não vaza para outros arquivos.



Anatomia de um Componente Vue

MeuComponente.vue

```
<script setup>
  // Lógica e imports aqui
</script>

<template>
  <h1>Olá IFMA</h1>
</template>

<style scoped>
  h1 { color: green; }
</style>
```

SCRIPT SETUP

O cérebro do componente. Onde importamos outros arquivos e definimos a lógica.

TEMPLATE

O corpo do componente. Onde escrevemos o HTML que será exibido na tela.

STYLE SCOPED

A roupa do componente. O CSS que afeta apenas este arquivo, sem vazar para os outros.

O Segredo do Style Scoped

`</>` No seu arquivo .vue

```
<style scoped>
h1 {
  color: #005C2E;
}
</style>
```

O atributo **scoped** avisa ao Vue que este estilo não deve "vazar" para outros componentes.

 O que o navegador recebe

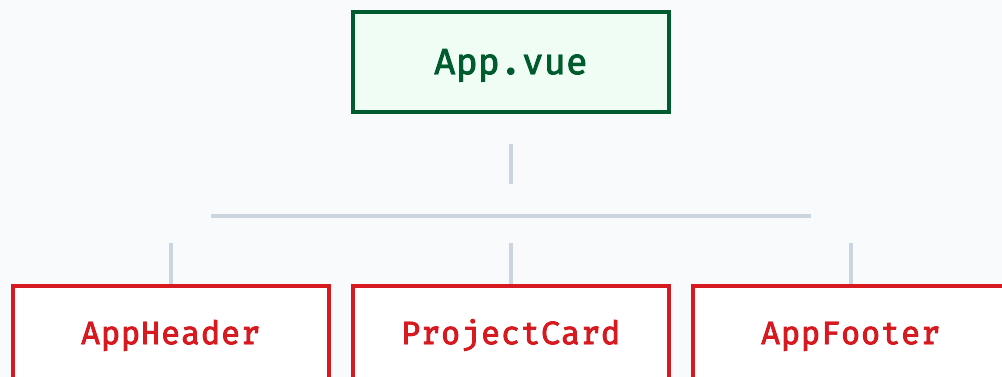
```
<h1 data-v-f3f3eg>
  Título do Projeto
</h1>

h1[data-v-f3f3eg] {
  color: #005C2E;
}
```

O Vue gera um **ID único** e o injeta nos elementos. Assim, o CSS só afeta quem tem esse ID específico.

A Árvore de Componentes

HIERARQUIA

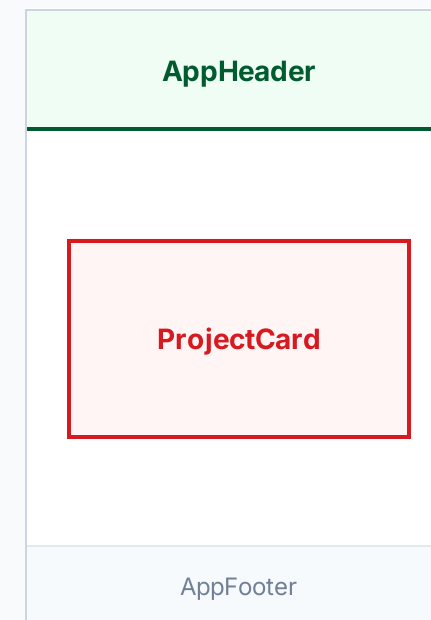


ORQUESTRAÇÃO

```
<template>
  <AppHeader />
  <main>
    <ProjectCard
  />
  </main>
  <AppFooter />
</template>
```

O componente **App.vue** decide a ordem e a posição de cada peça na tela.

RESULTADO FINAL



Interface montada

Estrutura de Pastas do Vite

EXPLORER

projeto/
 index.html
 package.json
 src/
 main.js
 App.vue

index.html

O ponto de montagem. Contém a `<div id="app">` onde o Vue será injetado.

src/main.js

O inicializador. Importa o Vue e o componente raiz para montar a aplicação.

src/App.vue

O componente raiz. É o "pai" de todos os outros componentes da sua interface.

Ponto de Entrada: main.js

src/main.js

```
import { createApp } from 'vue'
import App from './App.vue'

// Inicializa e monta
createApp(App).mount('#app')
```

1 IMPORTAÇÃO

Traz o motor do Vue e o componente raiz (App.vue) para o contexto do JavaScript.

2 CRIAÇÃO

A função **createApp** prepara a instância da aplicação usando o componente raiz como base.

3 MONTAGEM

O método **mount('#app')** injeta todo o sistema dentro da `<div id="app">` do seu `index.html`.

| Laboratório: Criando o Projeto



```
$ npm create vite@latest
```

SIGA A SEQUÊNCIA:

1. Project name: **prototipo-componentizado**
2. Select a framework: **Vue**
3. Select a variant: **JavaScript**

! Pré-requisitos

Certifique-se de que o Node.js está instalado e atualizado no laboratório.

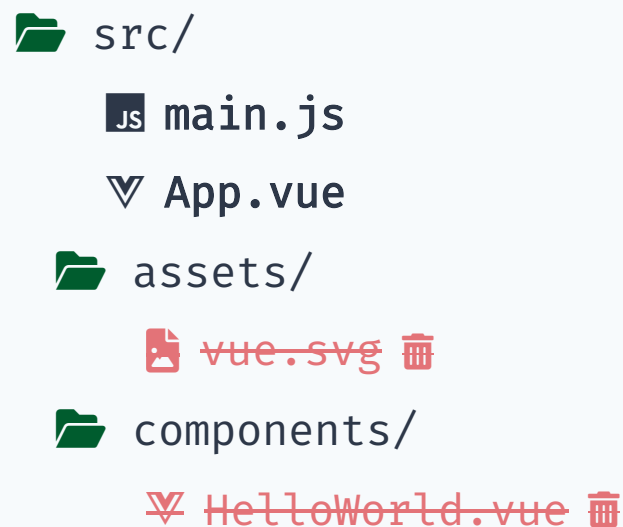
Node.js 20.19+ ou 22.12+

🗂 Dica de Teclado

Use as setas direcionais para navegar nas opções e **Enter** para confirmar.

| Laboratório: Limpeza e Preparação

EXPLORER



-  Remova o arquivo **components/HelloWorld.vue**.
-  Limpe o conteúdo de **App.vue** (deixe apenas as tags vazias).
-  Apague os ícones de exemplo na pasta **assets/**.
-  Remova o conteúdo padrão de **style.css**.



CUIDADO: Não apague o arquivo **main.js** nem o **index.html**. Eles são essenciais para a aplicação rodar.

Laboratório: Execução e HMR

>_ TERMINAL

```
$ npm run dev
```

```
VITE v5.4.0 ready in 128 ms
```

```
→ Local: http://localhost:5173/
```

```
→ Network: use --host to expose
```

```
press h + enter to show help
```

AÇÃO

Abra o navegador no endereço indicado pelo terminal (geralmente a porta 5173).

⚡ A MÁGICA DO HMR

Hot Module Replacement: Altere qualquer arquivo no VS Code, salve (Ctrl+S) e veja a mudança instantânea no navegador sem recarregar a página.

Mantenha o terminal aberto durante toda a aula.

Construção: AppHeader.vue

src/components/AppHeader.vue

```
<template>
  <header class="app-header">
    <p class="eyebrow">IFMA · Projeto
Integrador</p>
    <h1>Projeto Integrador Web</h1>
  </header>
</template>

<style scoped>
  .eyebrow { color: #1f8f3a; font-weight: 700; }
  h1 { color: #005c2e; }
</style>
```

AÇÃO

Crie o arquivo na pasta **components** e cole o código ao lado. Este componente cuidará apenas da identidade do topo.

RESULTADO ESPERADO

IFMA · Projeto Integrador
Projeto Integrador Web

Construção: ProjectCard.vue

src/components/ProjectCard.vue

```
<template>
  <article class="project-card">
    <h2>Problema observado</h2>
    <p>Como tornar as informações mais claras?
  </p>
    <button type="button">Marcar
interesse</button>
  </article>
</template>
<style scoped>
  .project-card { padding: 24px; background:
#f4f7f5; }
  button { background: #005c2e; color: white; }
</style>
```

RESPONSABILIDADE

Este componente é o núcleo do nosso protótipo. Ele exibe o problema que estamos tentando resolver e permite a interação do usuário.

! O botão permanece visual nesta aula. Não implementaremos o clique ainda.

Dividir para conquistar: cada peça no seu lugar.

Construção: AppFooter.vue

src/components/AppFooter.vue

```
<template>
  <footer class="app-footer">
    <small>Sistemas Web · IFMA</small>
  </footer>
</template>

<style scoped>
  .app-footer {
    margin-top: 24px;
    color: #666666;
  }
</style>
```

RESPONSABILIDADE

Exibir informações de encerramento e créditos do sistema de forma padronizada.

RESULTADO VISUAL

Desenvolvimento de Sistemas Web · IFMA

i O estilo scoped garante que este cinza não afete outros textos.

Montagem: Compondo no App.vue

src/App.vue

```
<script setup>
import AppHeader from
  './components/AppHeader.vue'
import AppFooter from
  './components/AppFooter.vue'
</script>
<template>
  <div class="page-shell">
    <AppHeader />
    <main>
      <ProjectCard />
    </main>
    <AppFooter />
  </div>
</template>
```

1 IMPORTAÇÃO

Os **imports** tornam os componentes filhos conhecidos pelo componente pai (App.vue).

2 USO COMO TAGS

Usamos as tags **PascalCase** no template para indicar onde cada peça deve aparecer.

3 ORQUESTRAÇÃO

O **App.vue** coordena a composição, decidindo a ordem e a estrutura da interface.

Identidade: Estilo Global e Reset

src/style.css

```
:root {  
  font-family: Arial, sans-serif;  
  background: #ffffff;  
}  
  
body {  
  margin: 0;  
  min-height: 100vh;  
}  
  
.page-shell {  
  width: min(90%, 720px);  
  margin: 0 auto;  
  padding: 32px 0;  
}
```

BASE COMUM

Define fontes e cores que serão herdadas por todos os componentes da aplicação.

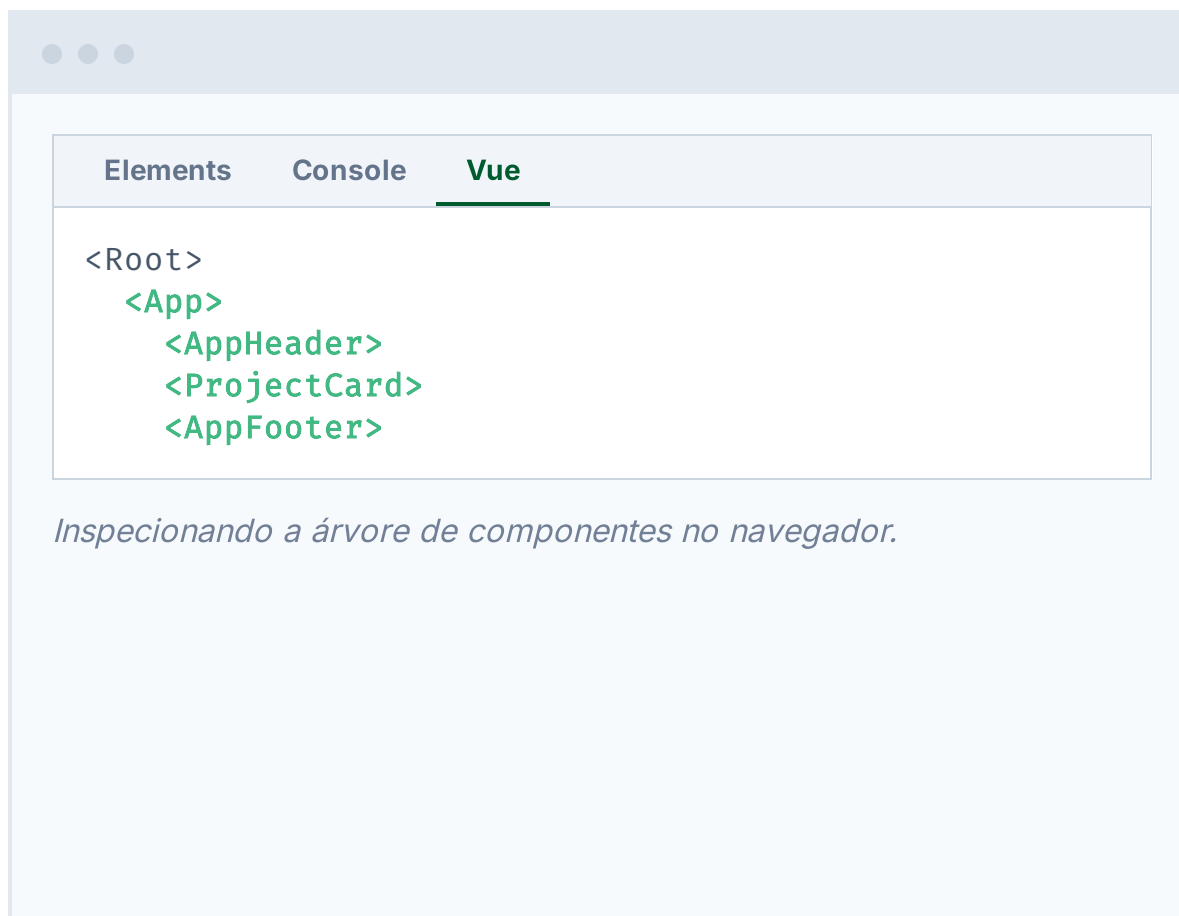
RESET DE MARGENS

Remove os espaços padrão do navegador para garantir controle total sobre o layout.

CENTRALIZAÇÃO

A classe **.page-shell** organiza o conteúdo no centro da tela, criando um "trilho" de leitura.

Investigação: Console e DevTools



O QUE VERIFICAR?

- ✓ **Renderização:** O componente aparece na aba "Vue" ou "Elements"?
- ✓ **Estilo:** O CSS do "scoped" está sendo aplicado corretamente?
- ✓ **Erros:** Há mensagens vermelhas na aba "Console"?

 **Atalho:** Pressione F12 ou Ctrl+Shift+I para abrir as ferramentas de desenvolvedor.

Versionamento: O Commit da Aula 2

>_ TERMINAL

```
$ git add .  
$ git commit -m "feat: estrutura inicial  
componentizada"
```

Verificando o histórico:

```
$ git log --oneline  
a1b2c3d feat: estrutura inicial  
componentizada  
e5f6g7h docs: documento de visão e  
requisitos
```

POR QUE COMMITAR AGORA?

Acabamos de realizar uma mudança estrutural profunda: saímos de um arquivo único para uma arquitetura de componentes.

MARCO DO PROJETO

Este commit garante que temos um ponto de restauração seguro antes de começarmos a adicionar lógica e dados dinâmicos na próxima aula.

 *Commits frequentes, sono tranquillo.*

| Definição de Pronto: O que entregamos?

Para considerar a Aula 2 concluída, seu projeto deve atender aos seguintes critérios técnicos:

AMBIENTE

- ✓ Projeto inicializado com Vite e template Vue (JavaScript).

COMPONENTIZAÇÃO

- ✓ Interface dividida em Header, Card e Footer no src/components.

ENCAPSULAMENTO

- ✓ Uso obrigatório de <style scoped> em todos os componentes.

EXECUÇÃO

- ✓ Servidor rodando (npm run dev) e sem erros no Console.

VERSIONAMENTO

- ✓ Commit realizado com a mensagem: "feat: protótipo componentizado".

IDENTIDADE

- ✓ Cores e tipografia seguindo o padrão institucional do IFMA.

✓ **Tudo pronto? Agora estamos preparados para a interatividade da Aula 3.**

Bilhete de Saída: Avaliação Individual

Responda mentalmente ou em seu log de aula antes de sair:

1 O que diferencia um Framework (como o Vue) de uma biblioteca comum de JavaScript?

2 Qual a principal vantagem técnica de utilizar o atributo "scoped" nas tags de estilo do Vue?

3 Por que a estratégia de componentização é vital para o crescimento e manutenção de sistemas web modernos?

 **TEMPO SUGERIDO: 3 MINUTOS**

Próximos Passos e Referências

▶▶ O QUE VEM NA AULA 3



Reatividade Profunda

Como o Vue monitora mudanças nos dados.



Diretiva v-model

Capturando dados de formulários em tempo real.



Eventos e Métodos

Fazendo o botão de interesse funcionar de verdade.



FONTES PARA ESTUDO



Guia Oficial do Vue.js 3

Documentação completa: vuejs.org



Documentação do Vite

Entenda o ferramental: vitejs.dev



MDN Web Docs

Referência para HTML, CSS e JS moderno.

Referências Bibliográficas

Fontes consultadas para os conceitos, ferramentas, comandos e padrões apresentados nesta aula.

L **VUE.JS.** Introduction. Vue.js Guide. Disponível em: <https://vuejs.org/guide/introduction.html>. Acesso em: 12 ago. 2026.

V **VITE.** Getting Started. Vite Guide. Disponível em: <https://vite.dev/guide/>. Acesso em: 12 ago. 2026.

M **MDN WEB DOCS.** HTML: HyperText Markup Language. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTML>. Acesso em: 12 ago. 2026.

N **NODE.JS.** Download and installation. Disponível em: <https://nodejs.org/en/download>. Acesso em: 12 ago. 2026.

n **NPM.** npm run-script. npm Docs. Disponível em: <https://docs.npmjs.com/cli/v10/commands/npm-run-script>. Acesso em: 12 ago. 2026.

F **FONT AWESOME.** Web Fonts with CSS. Disponível em: <https://fontawesome.com/docs/web/setup/host-your-own/webfonts>. Acesso em: 12 ago. 2026.

As referências aparecem aqui como apêndice; os conceitos correspondentes são trabalhados nos slides de Vue, Vite, Node/npm, HTML e ferramentas de desenvolvimento.